

■ EXAM REVISION SERIES

Trees & Hashing

Data Structures — Unit V Complete Revision Sheet

B.Tech 1st Year · 1st Semester

Subject: Data Structures (DS)

Last-Day Revision Ready

✦ *Concepts · Operations · Algorithms · Complexities · Flashcards · Exam Tips* ✦

1 Introduction to Trees

Definition

A **Tree** is a *non-linear, hierarchical* data structure consisting of nodes connected by edges. It has exactly one **root** node and zero or more **subtrees**.

Key Properties

- Number of edges = $n - 1$ (n = total nodes)
- Exactly one root (no parent)
- Every non-root node has exactly one parent
- Recursive structure — each subtree is also a tree
- No cycles allowed

Tree Diagram

```
A ← Root (Level 0) / \ B C ← Level 1 / \ D E
← Leaf nodes (Level 2)
```

Edges: A-B, A-C, B-D, B-E = 4 edges for 5 nodes ✓

Key Terminologies

Term	Definition	Example (above tree)
Root	Topmost node; no parent	A
Node	Basic unit holding data	A, B, C, D, E
Edge	Link between parent and child	A→B
Leaf Node	Node with no children	C, D, E
Internal Node	Node with at least one child	A, B
Height	Longest path from root to leaf	2
Depth of Node	Distance from root to that node	depth(D) = 2
Level	Set of all nodes at same depth	Level 1 = {B, C}
Subtree	A node + all its descendants	B's subtree = {B,D,E}
Degree	Number of children a node has	degree(B) = 2

2 Types of Trees

Binary Tree

Each node has at most **2 children** (left, right). General-purpose tree structure.

Binary Search Tree

Binary tree with **ordering: Left < Root < Right**. Enables fast search/insert/delete.

Full Binary Tree

Every node has **0 or 2** children. No node has exactly 1 child.

Complete Binary Tree

All levels filled except possibly the last; last level filled **left to right**. Used in heaps.

Balanced Tree (AVL)

Height difference of left & right subtrees ≤ 1 for every node. Guarantees $O(\log n)$ ops.

Skewed Tree

All nodes on one side (left/right skewed). Degenerates to linked list — $O(n)$ operations.

3 Binary Search Tree (BST)

BST Property

For every node N:

All keys in **Left Subtree** < N

All keys in **Right Subtree** > N

No duplicate keys (in standard BST)

Example BST

Insert: 50, 30, 70, 20, 40, 60, 80

```
50 / \ 30 70 / \ / \ 20 40 60 80
```

BST Node Structure

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

Applications

- Dictionary / spell checkers
- Auto-complete systems
- Range queries
- Sorted data retrieval
- Priority queues (when ordered)

4 BST Operations

Insertion

Algorithm (Step-by-step)

1. If tree is empty → create root node
2. Compare new key with current node
3. If new key < current → go to left child
4. If new key > current → go to right child
5. Repeat until NULL position found
6. Insert new node at NULL position

Time Complexity: $O(\log n)$ avg $O(n)$ worst (skewed)

Pseudocode

```
INSERT(root, key):
    if root == NULL:
        return newNode(key)
    if key < root.data:
        root.left = INSERT(root.left, key)
    elif key > root.data:
        root.right = INSERT(root.right, key)
    return root // key == root.data: duplicate
```

Searching

Algorithm

1. Start at root
2. If key == root → found!
3. If key < root → search left subtree
4. If key > root → search right subtree
5. If NULL reached → not found

Time Complexity: $O(\log n)$ avg $O(n)$ worst

```
SEARCH(root, key):
    if root == NULL:
        return NULL // Not found
    if key == root.data:
        return root // Found
    if key < root.data:
        return SEARCH(root.left, key)
    else:
        return SEARCH(root.right, key)
```

Deletion **IMPORTANT**

Three Cases for Deletion

Case 1: Leaf Node

Node has **no children**. Simply remove the node.

```
Delete 20: Before: After:
30 30 / \ / \ 20 40 x 40
```

Case 2: One Child

Replace node with its **only child**.

```
Delete 30: Before: After:
50 50 / / 30 20 / 20
```

Case 3: Two Children

Find **inorder successor** (smallest in right subtree). Copy its value. Delete the successor.

```
Delete 50: Inorder
successor = 60 Replace 50
with 60 Delete node 60
```

Deletion Pseudocode

```
DELETE(root, key):
    if root == NULL: return NULL

    if key < root.data:
        root.left = DELETE(root.left, key)
    elif key > root.data:
        root.right = DELETE(root.right, key)
    else: // Node found
        // Case 1 & 2: 0 or 1 child
        if root.left == NULL: return root.right
        if root.right == NULL: return root.left
        // Case 3: Two children
```

```

    successor = FIND_MIN(root.right) // inorder successor
    root.data = successor.data
    root.right = DELETE(root.right, successor.data)
    return root

FIND_MIN(node):
    while node.left != NULL:
        node = node.left
    return node

```

5 Tree Traversals (Very Important)

Example Tree

1 / \ 2 3 / \ 4 5

Inorder (LNR): 4 2 5 1 3
Preorder (NLR): 1 2 4 5 3
Postorder (LRN): 4 5 2 3 1
Level Order: 1 2 3 4 5

Memory Aid

Traversal	Order	Use Case
Inorder	Left → Node → Right	Sorted output for BST
Preorder	Node → Left → Right	Copy / Serialize tree
Postorder	Left → Right → Node	Delete tree, expressions
Level Order	BFS (queue-based)	Shortest path, level print

All traversals: $O(n)$ time · $O(h)$ space (h = height)

Inorder / Preorder / Postorder Pseudocode

```

INORDER(node):
    if node != NULL:
        INORDER(node.left)
        print(node.data)
        INORDER(node.right)

PREORDER(node):
    if node != NULL:
        print(node.data)
        PREORDER(node.left)
        PREORDER(node.right)

POSTORDER(node):
    if node != NULL:
        POSTORDER(node.left)
        POSTORDER(node.right)
        print(node.data)

```

Level Order (BFS) Pseudocode

```

LEVEL_ORDER(root):
    if root == NULL: return
    queue = []
    enqueue(queue, root)
    while queue not empty:
        node = dequeue(queue)
        print(node.data)
        if node.left != NULL:
            enqueue(queue, node.left)
        if node.right != NULL:
            enqueue(queue, node.right)

```

6 BST Complexity & Analysis

Operation	Best	Average	Worst (Skewed)
Search	$O(1)$	$O(\log n)$	$O(n)$
Insert	$O(1)$	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(\log n)$	$O(n)$
Traversal	$O(n)$ always		
Space	$O(n)$		

⚠ Skewed Tree Problem

When elements are inserted in sorted order (e.g., 10, 20, 30, 40), the BST degenerates into a linked list.

```
10 \ 20 \ 30 \ 40 ← O(n) search!
```

Solution: Use self-balancing trees (AVL, Red-Black) that maintain $O(\log n)$ always.

7 Introduction to Hashing

Definition

Hashing is a technique to map keys to specific locations (indices) in a **Hash Table** using a **Hash Function**.

Core Concept

- Key → Hash Function → Index (0 to $m-1$)
- Enables $O(1)$ average-case lookup
- Hash Table:** Array of size m

Basic Formula:

$$h(k) = k \% m$$

where k = key, m = table size

Real-World Analogies

📖 **Dictionary:** Word → page number (fast lookup, not sequential search)

📚 **Database Indexing:** Student ID → record location

🌐 **Web Cache:** URL → cached page in memory

Example

Key (k)	$h(k) = k \% 7$	Table Slot
15	$15 \% 7 = 1$	Slot 1
28	$28 \% 7 = 0$	Slot 0
43	$43 \% 7 = 1$	Slot 1 ← COLLISION!

8 Hash Functions

Division Method

$$h(k) = k \bmod m$$

Best when m is a **prime number** not close to a power of 2.
Simple and fast.

Example: $k=52, m=11 \rightarrow h(52) = 52 \% 11 = 8$

Multiplication Method

$$h(k) = \lfloor m \times (k \times A \bmod 1) \rfloor$$

$A \approx 0.618$ (golden ratio). Works for any m .
More complex but better distribution.

✅ Properties of a Good Hash Function

- Uniform distribution** — spread keys evenly across table
- Fast computation** — $O(1)$ time to compute $h(k)$
- Deterministic** — same key always maps to same index
- Minimize collisions** — few keys share same slot
- Use all of key** — consider all bits/characters

Load Factor (α)

$$\alpha = n / m \quad (n = \text{keys stored}, m = \text{table size})$$

Ideal: $\alpha \leq 0.7$ for open addressing

Collision: When two different keys hash to the *same index* in the hash table. Must be resolved for correct operation.

🔗 Method 1: Chaining (Separate Chaining)

Each table slot holds a **linked list** of all keys that hash to that index.

```
Table[0]: → 28 → NULL Table[1]: → 15 → 43 → NULL
Table[2]: → NULL Table[3]: → 17 → NULL
```

✅ Pros

- Simple to implement
- Table never full
- Handles high load factor

❌ Cons

- Extra memory (pointers)
- Cache unfriendly
- Worst: $O(n)$

🔑 Method 2: Open Addressing

All keys stored **within the table itself**. On collision, probe for next empty slot.

Linear Probing

$$h(k, i) = (h(k) + i) \bmod m$$

$i = 0, 1, 2, 3, \dots$ (probe sequence)

```
LINEAR_PROBE_INSERT(T, k):
    index = h(k)
    i = 0
    while T[index] != EMPTY:
        i = i + 1
        index = (h(k) + i) % m
        if i == m: raise "Table Full"
    T[index] = k
```

Quadratic Probing

$$h(k, i) = (h(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$$

Double Hashing

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

Best distribution, fewest clusters

10 Hash Table Operations

INSERT

1. Compute index = $h(\text{key})$
2. If slot empty → store key
3. Else → resolve collision
4. Store at new position

$O(1)$ avg **$O(n)$** worst

SEARCH

1. Compute index = $h(\text{key})$
2. If slot has key → found
3. Else follow probe/chain
4. Until key found or empty

$O(1)$ avg **$O(n)$** worst

DELETE

1. Compute index = $h(\text{key})$
2. Search for key
3. Mark slot as DELETED
4. Don't use NULL (breaks probing)

$O(1)$ avg **$O(n)$** worst

Search Pseudocode (Linear Probing)

```
HASH_SEARCH(T, k):
    index = h(k)
    i = 0
    while T[index] != EMPTY:
        if T[index] == k: return index // Found
        i = i + 1
        index = (h(k) + i) % m
        if i == m: break
    return -1 // Not found
```

11 Hashing Complexity Summary

Operation	Best Case	Average Case	Worst Case	Condition
Insert	$O(1)$	$O(1)$	$O(n)$	Worst = all keys collide
Search	$O(1)$	$O(1)$	$O(n)$	Worst = all in one chain
Delete	$O(1)$	$O(1)$	$O(n)$	Must search first
Space	$O(n)$ — proportional to stored elements			

12 Applications of Hashing

Application	How Hashing Helps
 Unique ID Generation	Hash values used as unique keys (UUID, checksums)
 Web Caching	URL → cached content ($O(1)$ lookup)
 Database Indexing	Primary key → record location (fast queries)
 Password Storage	Password hashed (SHA-256), stored hash compared
 Symbol Tables	Compiler maps variable names → memory addresses
 Deduplication	Hash content → detect duplicate files

Password Storage Flow

```
// On Registration:
hash = SHA256(password)
store(username, hash)

// On Login:
input_hash = SHA256(input_password)
if input_hash == stored_hash:
    return "Login successful"
```

Original password never stored — only its hash. Secure even if DB is leaked.

13 BST vs Hashing — Comparison

Feature	BST	Hash Table
Data Order	✓ Maintained (inorder = sorted)	✗ Not maintained
Search Time	$O(\log n)$ average	$O(1)$ average
Worst Case	$O(n)$ (skewed)	$O(n)$ (all collisions)
Data Structure	Tree (linked nodes)	Array
Range Queries	✓ Efficient	✗ Not supported
Memory	$O(n)$ + pointer overhead	$O(n)$ + load factor
Ordered Traversal	✓ Inorder traversal	✗ No ordering
Duplicate Handling	Typically not allowed	Allowed via chaining
Use Case	Sorted data, range queries	Fast lookup, cache, sets
Implementation	More complex	Simpler

14 Problem-Solving Patterns

BST Patterns

- **Insert sequence:** Follow left/right at each node
- **Delete (2 children):** Always use inorder successor (smallest in right subtree)
- **Validate BST:** Track min/max range per node
- **Height:** $1 + \max(\text{height}(L), \text{height}(R))$
- **kth smallest:** Inorder traversal, count k
- **LCA:** If both nodes < root → go left; if both > root → go right

Hashing Patterns

- **Collision check:** Always verify with example keys
- **Linear probing:** Wrap around using mod m
- **Deletion:** Mark DELETED (tombstone), never NULL
- **Load factor:** If $\alpha > 0.7 \rightarrow$ rehash (double table size)
- **Count frequency:** `hash[key]++` pattern
- **Two-sum problem:** Store complements in hash map

15 Key Terms & Definitions

Term	One-Line Definition
Tree	Hierarchical data structure with nodes and edges, no cycles
Root	Topmost node of a tree with no parent
Leaf	Node with no children (degree = 0)
Height	Longest path from root to any leaf node
Depth	Distance from root to a specific node
Subtree	A node and all its descendants
Binary Tree	Tree where each node has at most 2 children
BST	Binary tree with left < root < right ordering
Balanced Tree	Height difference of subtrees ≤ 1 at every node
Skewed Tree	All nodes on one side, degenerates to linked list
Inorder	Traversal: Left $\rightarrow$ Node $\rightarrow$ Right (gives sorted output for BST)
Preorder	Traversal: Node $\rightarrow$ Left $\rightarrow$ Right (used for tree copying)
Postorder	Traversal: Left $\rightarrow$ Right $\rightarrow$ Node (used for tree deletion)
Level Order	BFS traversal level-by-level using a queue
Inorder Successor	Smallest node in the right subtree; used in deletion
Inorder Predecessor	Largest node in the left subtree
Full Binary Tree	Every node has 0 or 2 children
Complete Binary Tree	All levels full except last; last filled left-to-right

Term	One-Line Definition
Hashing	Mapping keys to array indices via a hash function
Hash Function	Function that converts a key to a table index
Hash Table	Array-based data structure using hash function for lookup
Collision	Two keys mapping to the same hash table index
Chaining	Storing collided keys in a linked list at each slot
Open Addressing	Probing for next empty slot within the same table
Linear Probing	Sequential probing: check index+1, +2, +3, ... on collision
Quadratic Probing	Probe with quadratic increments: i^2 to reduce clustering
Double Hashing	Use second hash function to determine probe step size
Load Factor (α)	Ratio of stored elements to table size (n/m)
Rehashing	Resizing and rebuilding hash table when load factor is high
Clustering	Build-up of consecutive occupied slots in linear probing
Tombstone	DELETED marker placed at removed slot (open addressing)
Division Method	$h(k) = k \bmod m$ (simplest hash function)
Uniform Distribution	Keys spread evenly across all table slots
Symbol Table	Hash map used by compilers to store variable-address pairs
Degree of Node	Number of children of a node
Edge	Link/connection between a parent node and a child node

16 Formula & Syntax Reference Box

BST Key Formulas

```
// BST Insert Logic
if key < node: go LEFT
elif key > node: go RIGHT
```

Traversal Patterns

```
// Inorder (LNR)
IN(L)  $\rightarrow$  visit(N)  $\rightarrow$  IN(R)
```

Hashing Formulas

```
// Division Method
 $h(k) = k \% m$ 
```

```
else: duplicate (ignore)
```

```
// Height Formula
```

```
H = 1 + max(H_left, H_right)
```

```
// Nodes at Level L
```

```
max_nodes = 2^L
```

```
// Max nodes in BST height h
```

```
max = 2^(h+1) - 1
```

```
// Edges in any tree
```

```
edges = n - 1
```

```
// Preorder (NLR)
```

```
visit(N) → PRE(L) → PRE(R)
```

```
// Postorder (LRN)
```

```
POST(L) → POST(R) → visit(N)
```

```
// Level Order
```

```
Use Queue: BFS approach
```

```
// ALL: O(n) time
```

```
// Load Factor
```

```
 $\alpha = n / m$ 
```

```
// Linear Probing
```

```
 $h(k,i) = (h(k) + i) \% m$ 
```

```
// Quadratic Probing
```

```
 $h(k,i) = (h(k) + i^2) \% m$ 
```

```
// Double Hashing
```

```
 $h(k,i) = (h1(k) + i * h2(k)) \% m$ 
```

```
// If slot occupied:
```

```
apply probing → find empty
```

Q: What is a tree?

A: Non-linear hierarchical data structure with nodes and edges, no cycles

Q: What is the root of a tree?

A: Topmost node with no parent node

Q: What is a leaf node?

A: A node with no children (degree = 0)

Q: Edges in a tree with n nodes?

A: Exactly $n - 1$ edges

Q: BST property?

A: Left subtree < Root < Right subtree for every node

Q: Inorder traversal of BST gives?

A: Sorted (ascending) sequence of all keys

Q: Inorder traversal order?

A: Left → Node → Right (LNR)

Q: Preorder traversal order?

A: Node → Left → Right (NLR)

Q: Postorder traversal order?

A: Left → Right → Node (LRN)

Q: Level order traversal uses?

A: Queue-based BFS; visits nodes level by level

Q: BST deletion with two children?

A: Replace with inorder successor (smallest in right subtree)

Q: BST deletion with no children?

A: Simply remove the node (set parent pointer to NULL)

Q: BST deletion with one child?

A: Replace the node with its only child

Q: BST average search complexity?

A: $O(\log n)$ for balanced BST

Q: BST worst case complexity?

A: $O(n)$ for skewed (degenerate) tree

Q: What is a skewed tree?

A: BST where all nodes are on one side; behaves like linked list

Q: Full binary tree?

A: Every node has 0 or 2 children (never exactly 1)

Q: Complete binary tree?

A: All levels full except last; last level filled left to right

Q: What is hashing?

A: Technique to map keys to array indices using a hash function

Q: Division method hash formula?

A: $h(k) = k \bmod m$ (k = key, m = table size)

Q: What is a collision?

A: Two different keys mapping to the same hash table index

Q: Chaining collision resolution?

A: Linked list at each table slot to store multiple keys

Q: Linear probing formula?

A: $h(k,i) = (h(k) + i) \bmod m$

Q: Quadratic probing formula?

A: $h(k,i) = (h(k) + i^2) \bmod m$

Q: Double hashing formula?

A: $h(k,i) = (h_1(k) + i \times h_2(k)) \bmod m$

Q: Load factor formula?

A: $\alpha = n / m$ (n = elements, m = table size)

Q: Hash table average search time?

A: $O(1)$ with good hash function and low load factor

Q: Hash table worst case?

A: $O(n)$ when all keys hash to the same slot

Q: Why prime number for table size?

A: Reduces clustering and improves key distribution

Q: Inorder successor?

A: Smallest node in the right subtree of a given node

Q: Which traversal copies a tree?**Q: Which traversal deletes a tree?**

A: Preorder (process root before children)

A: Postorder (delete children before parent)

Q: BST maintains sorted order?

A: Yes — inorder traversal always yields sorted sequence

Q: Hash table maintains sorted order?

A: No — order of keys is not preserved

Q: When to prefer BST over Hashing?

A: When sorted order, range queries, or predecessors/successors needed

Q: Why use tombstone in deletion?

A: To avoid breaking probe sequences in open addressing

✗ Common Mistakes to Avoid

- **Wrong BST deletion:** Forgetting to handle all 3 cases; using inorder predecessor instead of successor without consistency
- **Traversal confusion:** Mixing up Inorder/Preorder/Postorder — memorize "LNR / NLR / LRN"
- **Hashing deletion:** Setting slot to NULL in open addressing breaks future searches — use DELETED marker
- **Skewed tree:** Claiming $O(\log n)$ for worst case when tree is skewed
- **Collision formula:** Forgetting to apply mod m in probing
- **Load factor:** Forgetting $\alpha = n/m$ and its impact on performance
- **Level numbering:** Root is at Level 0 (some books use Level 1 — be consistent)
- **Hash function:** Using table size as a power of 2 (poor for division method)

✓ Writing Tips for High Marks

- **Always draw diagrams** — BST insertion, deletion, traversal steps earn marks
- **Show step-by-step** — For BST insert, trace path from root to insertion point
- **Write clean pseudocode** — Use consistent indentation and show base cases
- **Mention complexity** — Always state $O(\log n)$ avg and $O(n)$ worst for BST
- **For traversals:** Write the order (LNR etc.) AND the actual output for the given tree
- **For hashing:** Show the hash table with indices 0 to $m-1$
- **Compare when asked:** Use a table for BST vs Hashing comparison
- **Mention applications** — Password hashing, symbol tables, caching = extra marks

★ 5-Mark Answer Structure:

Definition (1) → Diagram (1) → Algorithm/Pseudocode (1.5) → Complexity (0.5) → Example (1)

📄 BST Insertion — Exam Steps

1. Draw empty root → insert first element
2. Compare each new element with current node
3. Go left if smaller, right if larger
4. Show each comparison in the trace
5. Draw the final BST clearly

Example: Insert 50, 30, 70, 20, 40 into BST → trace each step

📄 Hashing — Exam Steps

1. State the hash function used
2. Draw table with indices 0 to $m-1$
3. Compute $h(k)$ for each key
4. Identify collisions clearly
5. Show probe sequence for each collision
6. State final table contents

★ Must-Know for Exam**2-Mark Topics**

- Define: tree, BST, hashing, collision
- BST property
- Types of traversals
- Hash function (division method)
- Collision resolution methods (names)

5-Mark Topics

- BST insertion with diagram
- BST deletion (all 3 cases)
- All 4 traversals with pseudocode
- Linear probing with example
- Chaining with diagram

10-Mark Topics

- Full BST with all operations
- All traversals on same tree
- Hash table construction + all operations
- BST vs Hashing with use cases
- Collision handling — compare all methods

🚀 No explanations. Pure facts. Read and remember.

🌳 **BST — Operations**

- Insert: Start root → go left (smaller) / right (larger) → insert at NULL
- Search: Same path as insert; return node if found, NULL if not
- Delete Case 1 (leaf): Remove directly
- Delete Case 2 (one child): Replace with child
- Delete Case 3 (two children): Replace with inorder successor → delete successor
- Inorder successor = FIND_MIN(node.right)

🔄 **Traversals**

- **Inorder (LNR)**: Left → Node → Right → sorted output
- **Preorder (NLR)**: Node → Left → Right → copy tree
- **Postorder (LRN)**: Left → Right → Node → delete tree
- **Level Order**: BFS with queue → level-by-level
- All traversals: **O(n)** time

📊 **BST Complexities**

Op	Avg	Worst
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

Worst = skewed tree (sorted insertion)

🔑 **Hashing — Formulas**

- **Division**: $h(k) = k \% m$
- **Load factor**: $\alpha = n / m$ (keep ≤ 0.7)
- **Linear probe**: $h(k,i) = (h(k) + i) \% m$
- **Quadratic**: $h(k,i) = (h(k) + i^2) \% m$
- **Double hash**: $h(k,i) = (h_1(k) + i \cdot h_2(k)) \% m$
- Delete in open addressing → use DELETED tombstone
- Best m = prime number

📊 **Hashing Complexities**

Op	Avg	Worst
Insert	$O(1)$	$O(n)$
Search	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

⚖️ **BST vs Hashing**

Feature	BST	Hash
Order	✅ Yes	❌ No
Search	$O(\log n)$	$O(1)$
Range Query	✅ Yes	❌ No
Use Case	Sorted data	Fast lookup

💡 **Memory Tricks**📌 **Traversal Mnemonics**

- **Inorder** → **In** sorted order
- **Preorder** → **Pre**serve (copy) tree
- **Postorder** → **Post**-cleanup (delete)
- NLR / LNR / LRN: N position = order of root visit

📌 **BST Deletion Memory**

- 0 child → Just remove
- 1 child → Bypass node
- 2 children → Inorder Successor saves the day
- IS = smallest in RIGHT subtree

📌 **Hash Collision Order**

- Linear → Simple but clusters
- Quadratic → Less cluster
- Double → Best distribution
- Chaining → Never full

★ **Top 5 Must-Draw Diagrams:**

1. BST after inserting given sequence
2. BST deletion showing all 3 cases
3. Inorder/Preorder/Postorder on same tree
4. Hash table with chaining (collision shown)
5. Linear probing table with probe sequence

★ **Top 5 Formulas to Remember:**

1. $h(k) = k \% m$ — Division method
2. $\alpha = n / m$ — Load factor
3. $h(k,i) = (h(k) + i) \% m$ — Linear probing
4. Edges = $n - 1$ — Any tree
5. Height = $1 + \max(H_{\text{left}}, H_{\text{right}})$ — Recursive

